TRABAJO PRÁCTICO GRUPAL – Parte 1

Software embarcado de una nave interestelar tripulada

Julieta Fontana

Joaquín Pollio

Simón Francano Sanchez

Azul García Garro

Índice

1. Introducción

2. Organización del código

3. Patrones de diseño

4. Reglas de negocio y validaciones

5. Flujo de operación, misiones e informes

6. Conclusiones

1. Introducción

El presente trabajo tiene como objetivo diseñar e implementar un sistema de gestión y operación de naves espaciales, aplicando los conceptos de programación orientada a objetos y los patrones de diseño estudiados durante la cursada.
Para su desarrollo se utilizan los patrones de diseño Factory, State, Template Method y Decorator, con el propósito de distribuir adecuadamente las responsabilidades, favorecer la reutilización de código y facilitar la incorporación de nuevas funcionalidades. También se aplican principios de encapsulamiento, abstracción, herencia, polimorfismo y diseño por contrato, estableciendo restricciones que permiten mantener la consistencia del sistema.

2. Organización del código

El proyecto se desarrolla en Java y sigue un enfoque de programación orientada a objetos. Su organización se basa en la separación de responsabilidades mediante paquetes, clases e interfaces, procurando reducir el acoplamiento entre los distintos componentes.

Los principales paquetes del sistema son:

- **naves:** contiene las abstracciones y clases concretas correspondientes a los distintos tipos de naves, junto con la fábrica encargada de su creación.
- **motor:** contiene la implementación del motor y sus distintos estados operativos, utilizando el patrón State para gestionar las transiciones.
- **tripulacion:** agrupa las clases relacionadas con los tripulantes, sus cargos y sus planetas de origen.
- **tripulacion.haberes:** reúne los componentes responsables del cálculo de haberes y la aplicación de adicionales mediante el patrón Decorator.
- **misiones:** contiene las clases correspondientes a las misiones espaciales, su creación y los informes asociados.
- **asistente:** incluye el componente encargado de gestionar la comunicación y las órdenes dirigidas a una nave.
- **bitacora:** contiene el componente destinado al registro de acontecimientos del sistema.

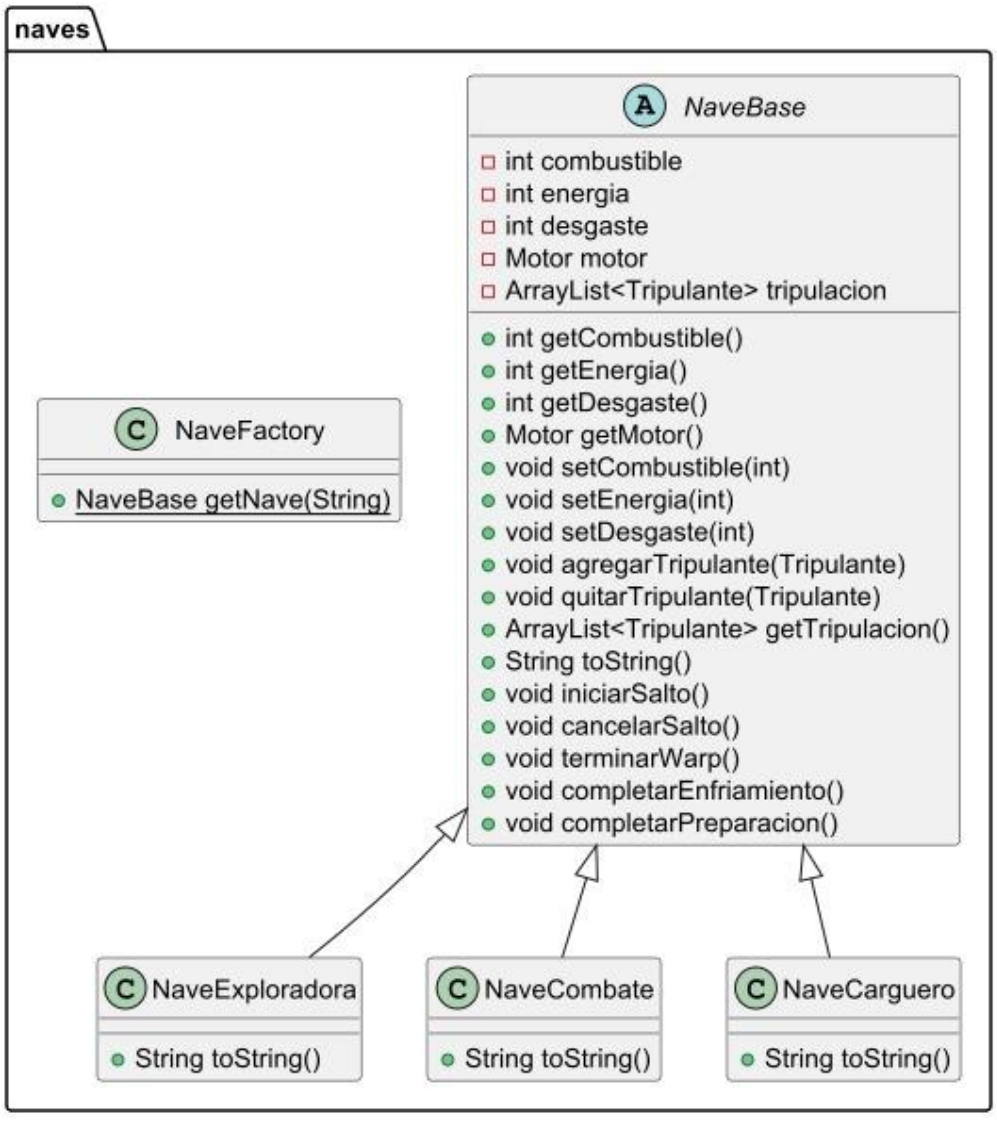
Esta distribución permite organizar el código según la responsabilidad de cada componente y facilita su mantenimiento. Asimismo, el uso de interfaces y clases abstractas permite definir comportamientos comunes y especializarlos en las clases concretas.

3.Diagramas UML

Se presentan a continuacion los diagramas de clases UML de la Entrega 1, organizados por paquete. Cada diagrama muestra las clases del paquete, sus atributos y métodos principales, y las relaciones entre ellas (herencia, implementación de interfaces y composición). Donde corresponde, se indica el patrón de diseño aplicado.

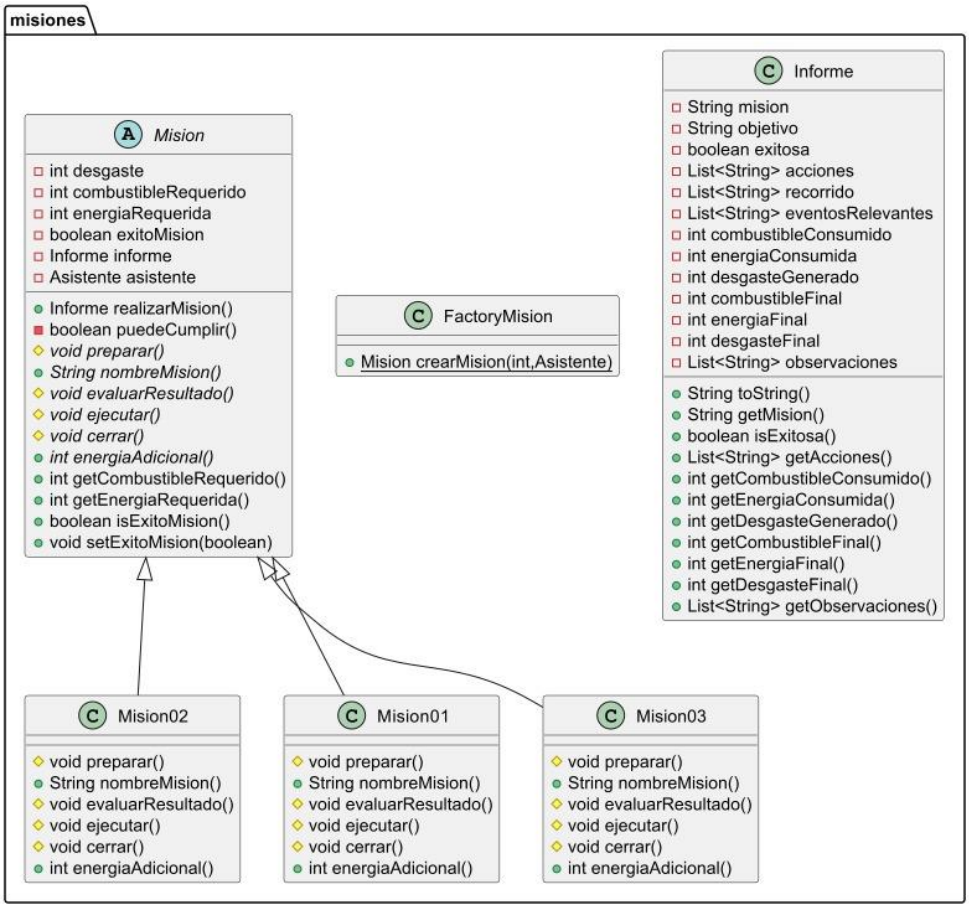
- **Paquete naves**.

NaveBase es la clase abstracta común a los tres tipos de nave: NaveExploradora, NaveCarguero y NaveCombate. Las naves se crean mediante NaveFactory (patrón Factory), así que el código cliente no instancia directamente las clases concretas. Cada nave tiene su motor, sus recursos y su tripulación.



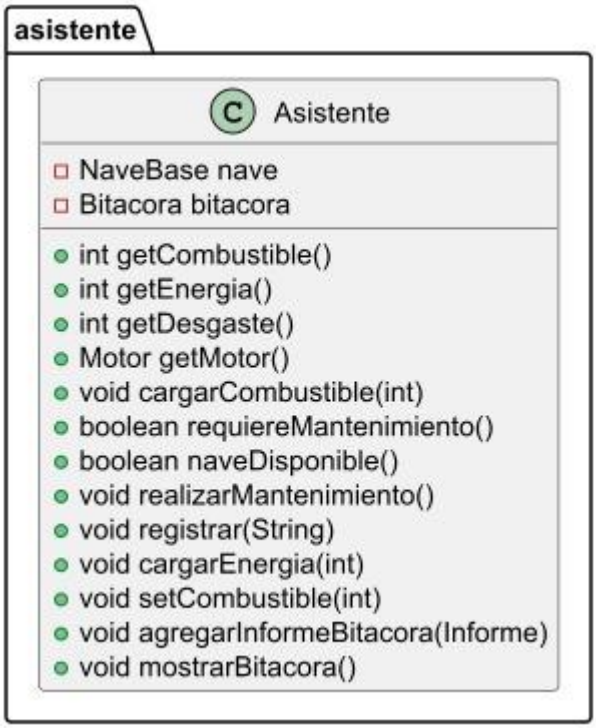
- Paquete misiones.

Mision define en realizarMision() el ciclo general de toda misión (preparar, ejecutar, evaluar y cerrar), y Mision01, Mision02 y Mision03 implementan cada paso (patrón Template Method). Las misiones se crean con FactoryMision, y al cerrarse generan un Informe.



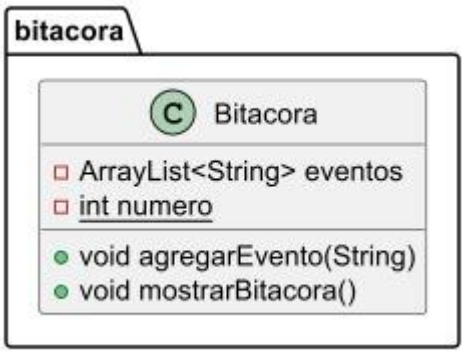
-Paquete asistente.

El Asistente recibe las órdenes y las coordina con la nave, sin implementar el comportamiento interno de los subsistemas. Además registra los resultados en la bitácora.



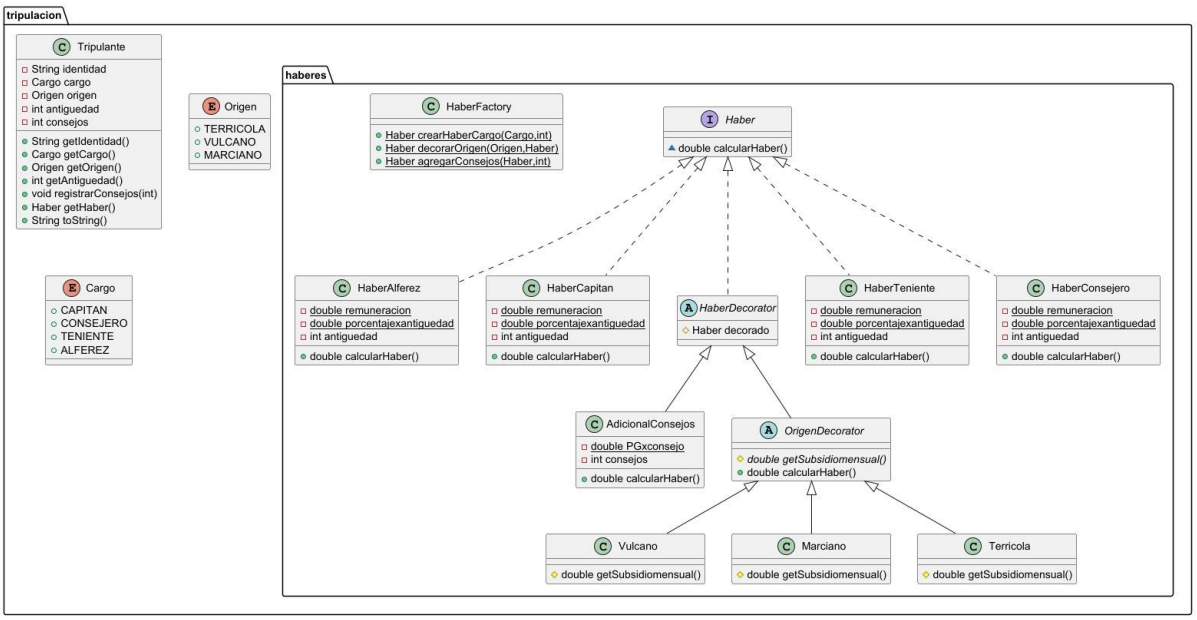
-Paquete bitácora.

La Bitácora registra en orden temporal los eventos relevantes del sistema.



-Paquete tripulación.

Tripulante representa a cada integrante con su cargo y origen. El haber mensual se calcula en el subpaquete haberes con el patrón Decorator. Los haberes de cada cargo implementan la interfaz Haber, y los decoradores (AdicionalConsejos y los orígenes, a través de OrigenDecorator) envuelven un Haber y le suman su concepto. Así se combinan cargo, origen y consejos sin crear una clase por cada combinación.



4. Patrones de diseño

Durante el desarrollo del sistema se aplicaron distintos patrones de diseño orientados a resolver problemas específicos de creación de objetos, organización de comportamientos y distribución de responsabilidades.

Los principales patrones utilizados son **Factory, State, Template Method y Decorator**. Su implementación permite organizar el código en componentes con responsabilidades definidas, reducir dependencias entre clases y facilitar futuras modificaciones o extensiones del sistema.

Factory: creación de naves y componentes

NaveFactory concentra la elección entre NaveExploradora, NaveCarguero y NaveCombate. El código cliente solicita el tipo y recibe la abstracción NaveBase. También existen FactoryMision, que selecciona misiones por número, y HaberFactory, que selecciona la remuneración base según el cargo y agrega decoraciones por origen o consejos.

```
NaveBase nave = NaveFactory.getNave("Exploradora");
```

State: estados del motor Warp

Motor conserva una referencia State y delega operaciones como iniciarSalto, cancelarSalto, completarPreparacion, terminarWarp y completarEnfriamiento. Los estados concretos encapsulan las respuestas y transiciones correspondientes. El constructor de Motor establece el estado inicial DisponibleState. La delegación permite separar el comportamiento según el estado del motor.

Template Method: ciclo de misión

Mision define el método final realizarMision(), que establece el orden general de ejecución: verificar recursos y disponibilidad, preparar, ejecutar, evaluar el resultado y cerrar. Las clases Mision01, Mision02 y Mision03 especializan los pasos mediante métodos sobrescritos.

```
public final Informe realizarMision() {
     if (puedeCumplir()) {
         preparar(); ejecutar(); evaluarResultado(); cerrar();
     }
     return this.informe;
 }
```

Estado actual: los métodos preparar y ejecutar están vacíos en las tres misiones; cerrar devuelve null y el atributo informe no recibe una asignación observable. Por ello, el ciclo está estructurado, pero el resultado de misión aún no está integrado completamente.

Decorator: composición de haberes

Haber define el contrato calcularHaber(). Las clases HaberCapitan, HaberConsejero, HaberTeniente y HaberAlferez calculan una remuneración base según cargo y antigüedad. HaberDecorator mantiene una referencia Haber, mientras OrigenDecorator y sus subclases Terricola, Vulcano y Marciano agregan componentes vinculados al origen. AdicionalConsejos agrega un concepto adicional por cantidad de consejos.

El patrón permite combinar responsabilidades de liquidación sin multiplicar subclases para cada combinación de cargo, origen y adicionales.

5. Reglas de negocio y validaciones

Asistente consulta los recursos y ofrece operaciones para cargar combustible o energía, consultar disponibilidad y efectuar mantenimiento. Las cargas comparan el total con el máximo de 100; el mantenimiento se considera necesario cuando el desgaste es al menos 80 y su ejecución lo restablece a cero. Los eventos de carga y mantenimiento se registran en Bitacora.

Aspectos por completar o fortalecer: los setters públicos de NaveBase asignan valores sin validar rangos; las cargas del asistente no rechazan explícitamente cantidades negativas; el desgaste tampoco se valida en su setter. Por lo tanto, el código actual no garantiza en todos los puntos de acceso las invariantes de recursos no negativos y desgaste entre 0 y 100.

En HaberFactory sí se emplea IllegalArgumentException cuando el cargo u origen recibido es nulo o no está soportado. En el motor, varias operaciones inválidas

comunican el rechazo por salida estándar; no todas se modelan mediante excepciones.

6. Flujo de operación, misiones e informes

Cada Asistente recibe una NaveBase en su constructor y crea una Bitacora propia. El asistente centraliza consultas y órdenes de recursos, aunque NaveBase conserva métodos públicos que permiten operar directamente el motor y modificar recursos. La exclusividad de acceso a través del asistente, planteada en la consigna, requiere revisión adicional.

Mision utiliza el asistente para consultar combustible, energía y disponibilidad antes de iniciar su secuencia. La clase Informe ya define datos de identificación, objetivo, éxito, acciones, recorrido, eventos relevantes, consumos, estado final y observaciones. Sin embargo, la construcción y devolución efectiva de informes desde las misiones está pendiente.

7. Conclusiones

El desarrollo de este trabajo práctico permitió aplicar los conceptos de programación orientada a objetos y patrones de diseño estudiados durante la cursada de Programación III, mediante la construcción de un sistema de gestión y operación de naves espaciales.

A lo largo del proyecto se buscó establecer una estructura modular, separando las responsabilidades de los distintos componentes y utilizando abstracciones que favorezcan la reutilización y el mantenimiento del código. La aplicación de los patrones Factory, State, Template Method y Decorator permitió abordar diferentes necesidades del sistema, como la creación de objetos, la administración de estados, la organización de algoritmos y la incorporación de funcionalidades adicionales.

Asimismo, se consideraron las restricciones y reglas de negocio establecidas en el enunciado, procurando mantener la consistencia de los recursos y las operaciones de las naves.

En conclusión, esta primera entrega permitió establecer las bases del sistema y poner en práctica los conocimientos adquiridos, dejando una estructura preparada para continuar desarrollando, integrando y mejorando las funcionalidades en las próximas etapas del trabajo práctico.